Micro:bit LED Animation Project

Project Report | ARM Assembly

Course	Computer Architecture
Team Members	Kshitij Pachpande – BC2025049
	Dhyan Patel – BC2025072
	Rushil Parikh – IC2025025
	Lakshya Raval – BC2025054

1. What This Project Is About

For this assignment, we made a simple LED animation display on the Micro:bit. The board has a 5x5 grid of 25 small LEDs, and we programmed it to show 5 different moving patterns on those LEDs. The user can switch between the patterns using the two buttons on the board (Button A and Button B).

The whole thing is written in ARM Assembly language, which is basically the most basic level of programming you can do — there's no Python, no C, no libraries. We're directly telling the hardware what to do, pin by pin.

In short, the project does three things:

- Shows 5 animated patterns on the LED grid, each with 4 frames that loop
- Button A goes to the previous pattern, Button B goes to the next one
- Handles button presses cleanly so one press = one action (no double-jumping)

2. How We Built It

The LED Grid

The Micro:bit LED grid works in a slightly odd way — you don't control each LED individually. Instead, there are 5 row wires and 5 column wires. To light up an LED, you turn on one row and turn off specific columns (the columns are "active low", meaning LOW = ON and HIGH = OFF). We loop through each row very quickly, one at a time, and because it cycles so fast, your eyes see all LEDs lit at once. This trick is called multiplexing.

The 5 Patterns

We stored all 5 patterns directly in the code as raw numbers. Each pattern has 4 frames, and each frame is just 5 bytes (one per row). The patterns are:

- Expanding Square — a dot in the middle that grows into a square, then shrinks back
- Horizontal Scan Line — a full row of LEDs sweeping downward, like a radar screen
- Spinning Line — a line that rotates through different angles
- Beating Heart — a heart shape that pulses big and small
- Raindrop — a single dot that falls from top to bottom

The Buttons

Button A (pin 14) and Button B (pin 23) are connected to the GPIO pins on the chip. We set them up with pull-up resistors, which means they read as HIGH (1) when not pressed, and LOW (0) when pressed. To detect a button press, we compare the current reading with the last reading — if it was 0 before and is 1 now, that's a new press. This way one physical press only counts once.

Timing

We use simple delay loops to control timing. Each row stays on for about 2,000 clock cycles during scanning, and we wait about 60,000 cycles between animation frames. There's no real clock or timer chip involved — just the CPU counting loops.

3. Problems We Ran Into

LEDs showing the wrong pattern (inverted)

When we first tested it, all the LEDs that should be OFF were ON, and vice versa. We forgot that the columns are active-low — a 1 in our pattern data doesn't mean "turn on", it means "turn off". We fixed this by flipping all the bits before sending them to the column pins.

Patterns getting scrambled mid-animation

The animation was jumping around randomly after the first frame. The problem was that we were keeping the current frame number in a register (r1), but when we called other functions, those functions were overwriting r1 with their own values. The fix was to save the frame number into r7 right at the start of the drawing function, since r7 is a "safe" register that other functions are not supposed to touch.

One button press skipping multiple patterns

Pressing Button B once was jumping 2 or 3 patterns forward. This happens because buttons physically bounce — the metal contacts make and break contact many times in a millisecond when you press them. We added a small delay (about 300,000 cycles) after reading the button, and also made sure the code only reacts when the button goes from "not pressed" to "pressed" (not while it's being held down).

Math without multiply instructions

To find where a pattern's data starts in memory, we needed to multiply by 20 (for the pattern) and by 5 (for the frame). ARM Thumb Assembly doesn't have a convenient multiply-by-constant instruction, so we had to get creative using shifts and adds. For example, x × 5 becomes (x × 4) + x, which is just a left-shift and an add.

4. Results

Everything works as expected. All five patterns display correctly and the animations look smooth. Switching between patterns with the buttons works reliably with no accidental double-presses. The code runs on bare hardware with no operating system or libraries — just our Assembly code talking directly to the chip.

We tested all five patterns and both button directions. Boundary conditions also work — pressing Button A on pattern 0 does nothing, and pressing Button B on pattern 4 does nothing.

5. What Could Be Improved

- Add more patterns — the format is simple, so adding new ones is easy
- Use hardware timers instead of delay loops — this would make animations smoother and also let buttons respond faster
- Use the accelerometer on the board — you could shake the Micro:bit to shuffle patterns, or tilt it to move things around
- Speed control — hold a button to make the animation faster or slower
- Custom patterns via Bluetooth or serial — send your own pattern from a phone or computer

6. Conclusion

This project was a good hands-on experience with low-level programming. Writing in Assembly made us think about things like how memory works, how GPIO pins are controlled, and how to work around hardware quirks like active-low signals and button bounce. It took some debugging to get right, but the final result works well and covers all the project requirements.